

Assignment 1

Observability Report

Project: Ticket Booking Observability

Course: Enterprise Software Development, Fall 2026

Part A: Project

Problem. Manual event ticket booking systems give operators no visibility into how the system behaves under load or failure: whether requests are slow, which bookings are failing, or why. This project builds a minimal ticket booking API and instruments it fully, so that its behaviour, both normal and faulty, can be observed, measured, and explained rather than guessed at.

Intended users. Backend/SRE-style operators who need to monitor a booking service's health, and, indirectly, end users who benefit from a system whose problems can be diagnosed and fixed quickly.

Solution. A FastAPI service (ticket-app) exposes three core endpoints (/events, /book, /cancel/{id}) backed by in-memory data. It is instrumented with all four Prometheus metric types, visualised in Grafana, and produces structured JSON logs shipped through Filebeat into Elasticsearch and searched via Kibana. Two fault-injection endpoints allow reproducing a slow-dependency incident and a metric cardinality explosion on demand.

How to try it. See README.md for full setup instructions. In short: docker compose up -d --build, then visit <http://localhost:8000/docs> to use the app, <http://localhost:3000> for the Grafana dashboard, and <http://localhost:5601> for log search.

Part B: Metrics

Setup

Prometheus (v2.48.0) scrapes the app every 5 seconds via `telemetry/prometheus/prometheus.yml`. Grafana (v10.2.0) is auto-provisioned with the Prometheus datasource and visualises everything on a single dashboard, **Ticket Booking War Room**.

Metric catalog

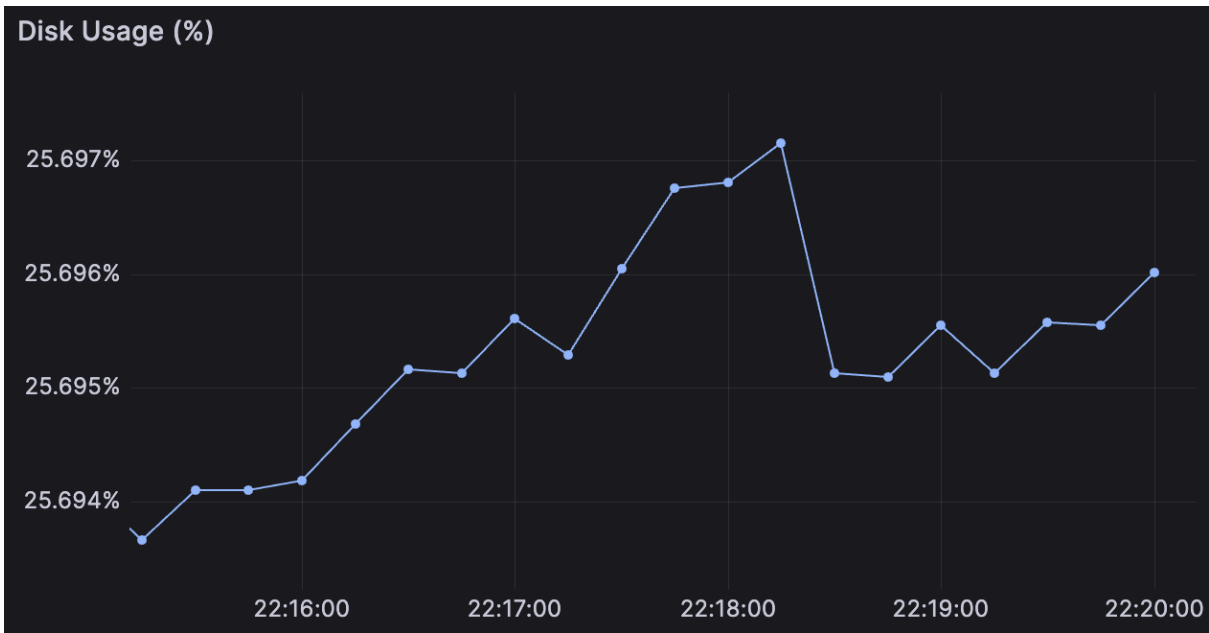
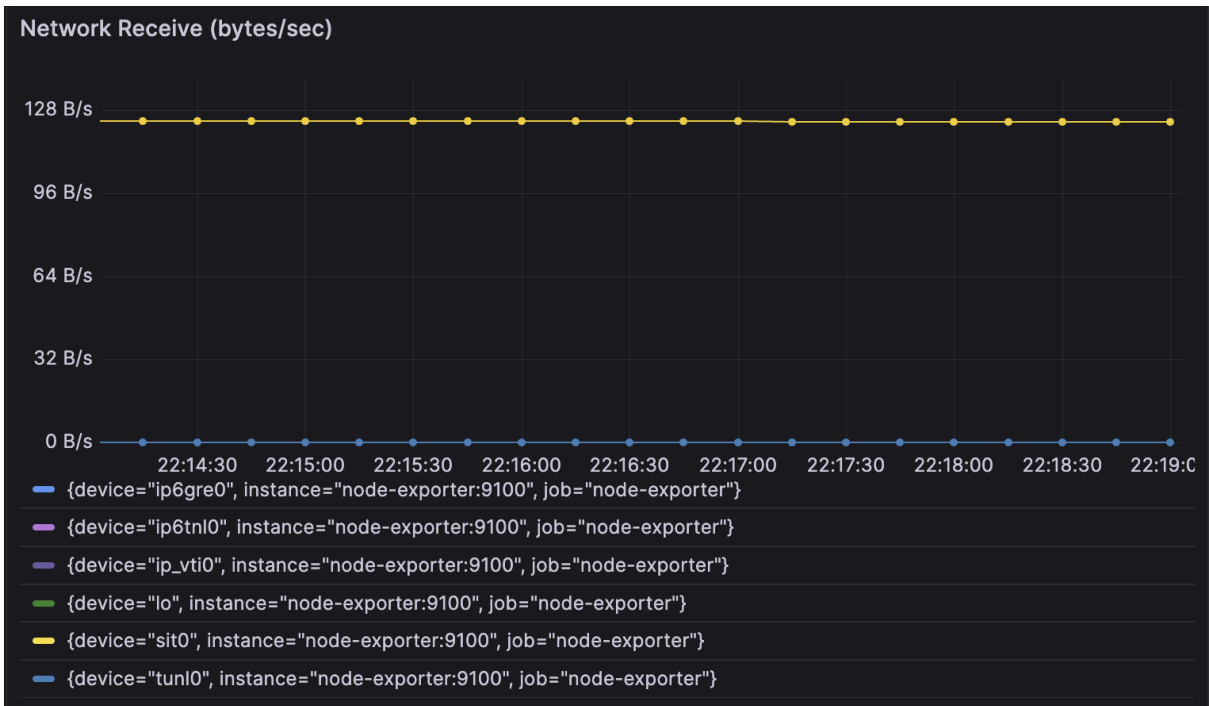
Metric	Type	Purpose	Unit	Labels	Recorded in code
http_requests_total	Counter	Total HTTP requests, rate and error tracking	count	method, path, status	track_requests middleware, every request
http_request_duration_seconds	Histogram	Request latency distribution (for P95)	seconds	method, path	track_requests middleware, every request
tickets_booked_total	Counter	Business metric, tickets sold	count	event_id	inside /book, on successful booking
bookings_cancelled_total	Counter	Business metric — cancellations	count	none	inside /cancel/{id}, on successful cancel
seats_currently_booked	Gauge	Live seats booked across all events	count	none	incremented in /book, decremented in /cancel
booking_processing_seconds	Summary	Average time to process a booking	seconds	none	@booking_processing_seconds.time() decorator on /book
demo_requests_total	Counter (demo only)	Cardinality-explosion demonstration	count	request_id (unbounded, deliberate)	inside /book, only when cardinality_bomb_enabled
node_cpu_seconds_total, node_memory_*, node_filesystem_*	Various	Infra metrics (CPU, memory, disk)	seconds / bytes	exported by Node Exporter	Node Exporter, not app code

This covers all four required metric types (Counter, Gauge, Histogram, Summary), both application metrics (rate, errors, duration) and business metrics (tickets booked, cancellations), plus infrastructure metrics.

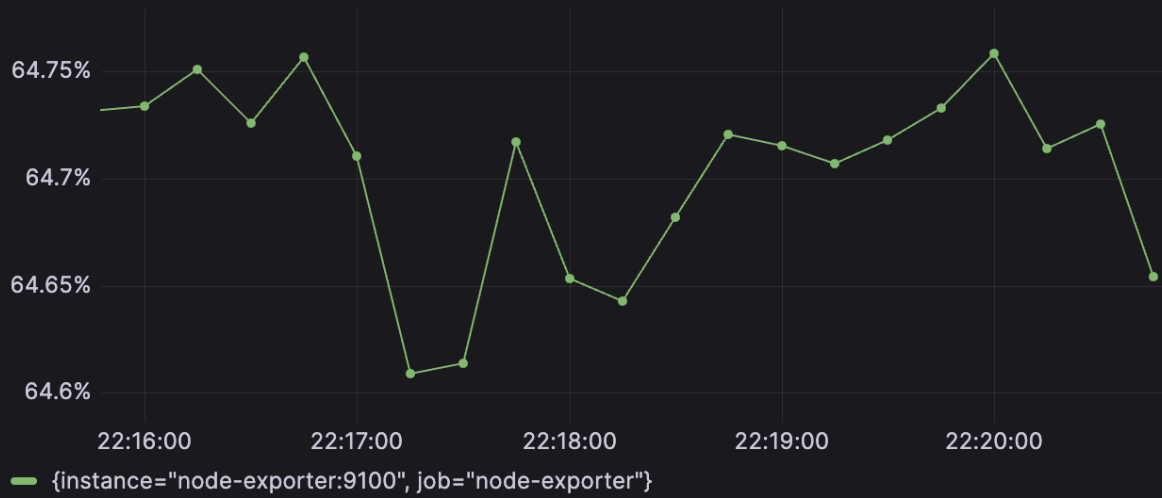
Panel	Query	What it shows
Tickets Booked (Total)	<code>tickets_booked_total</code>	Raw counter per event, proof of the metrics pipeline working end to end
Request Rate (req/sec)	<code>sum(rate(http_requests_total[1m]))</code>	Application throughput
Error Rate (%)	<code>sum(rate(http_requests_total{status=~"4..5.."}[1m])) / sum(rate(http_requests_total[1m])) * 100</code>	Percentage of failing requests
Latency P95 (ms)	<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le)) * 1000</code>	95th-percentile response time
Seats Currently Booked	<code>seats_currently_booked</code>	Business gauge, rises and falls with bookings/cancellations
Avg Booking Time (ms)	<code>rate(booking_processing_seconds_sum[5m]) / rate(booking_processing_seconds_count[5m]) * 1000</code>	Summary-derived average processing time
CPU Usage (%)	<code>100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)</code>	Host CPU utilisation
Memory Usage (%)	<code>(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100</code>	Host memory utilisation
Disk Usage (%)	<code>(1 - (node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/" })) * 100</code>	Host disk utilisation

Dashboard panels and PromQL

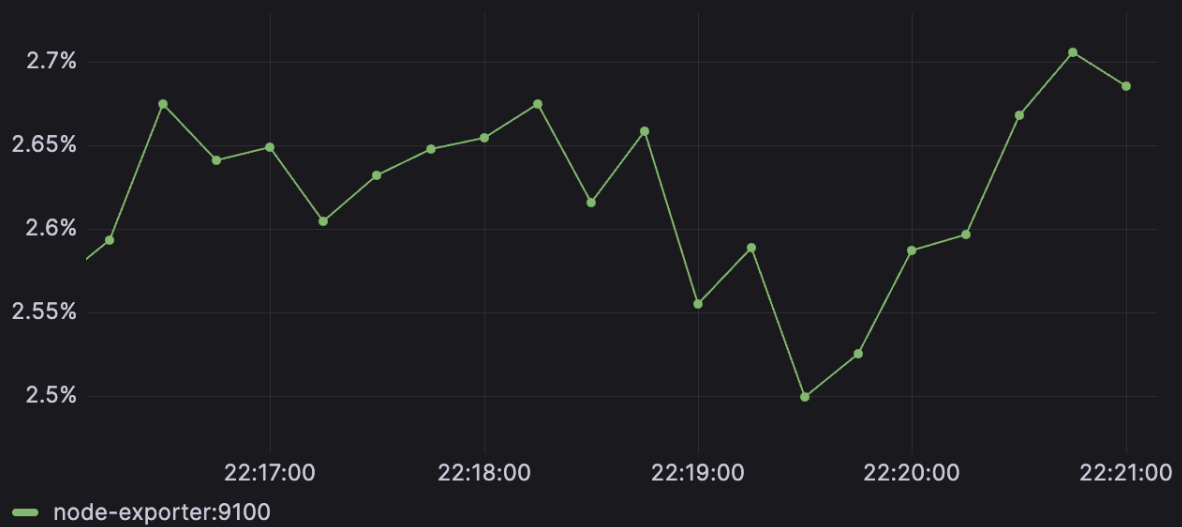
Attached are screenshots of all dashboard panels.



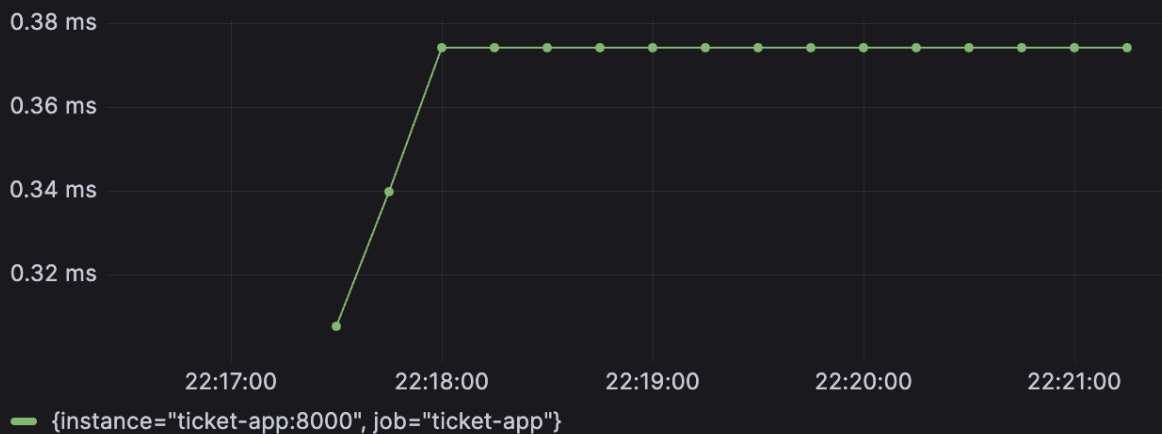
Memory Usage (%)



CPU Usage (%)



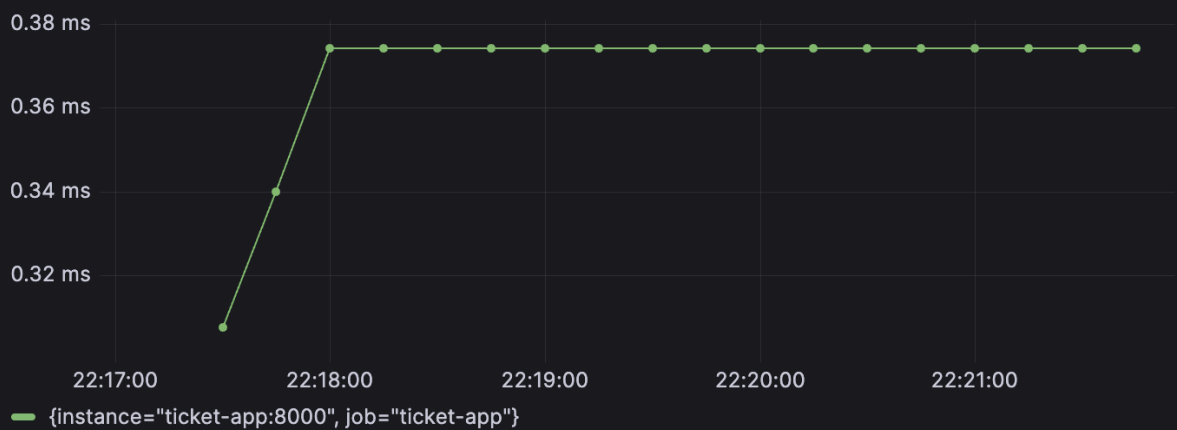
Avg Booking Time (ms)



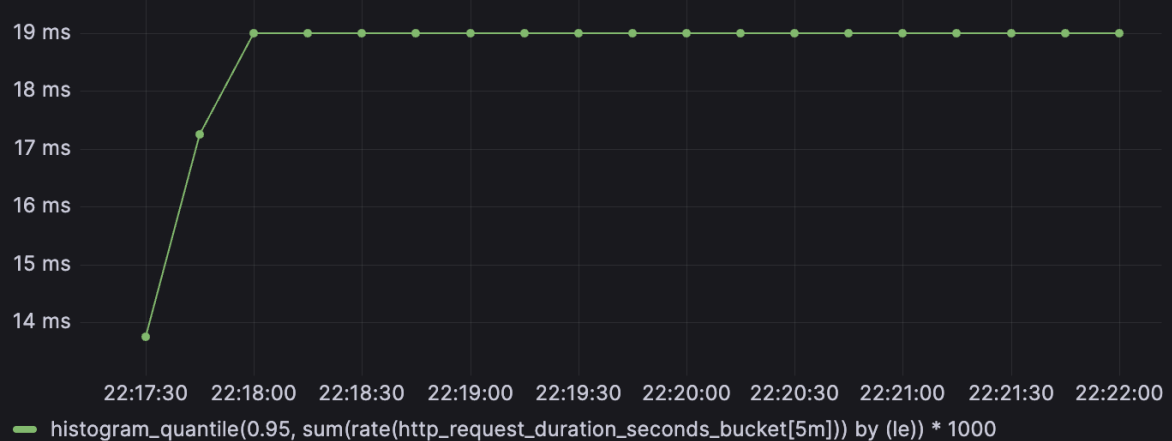
Seats Currently Booked

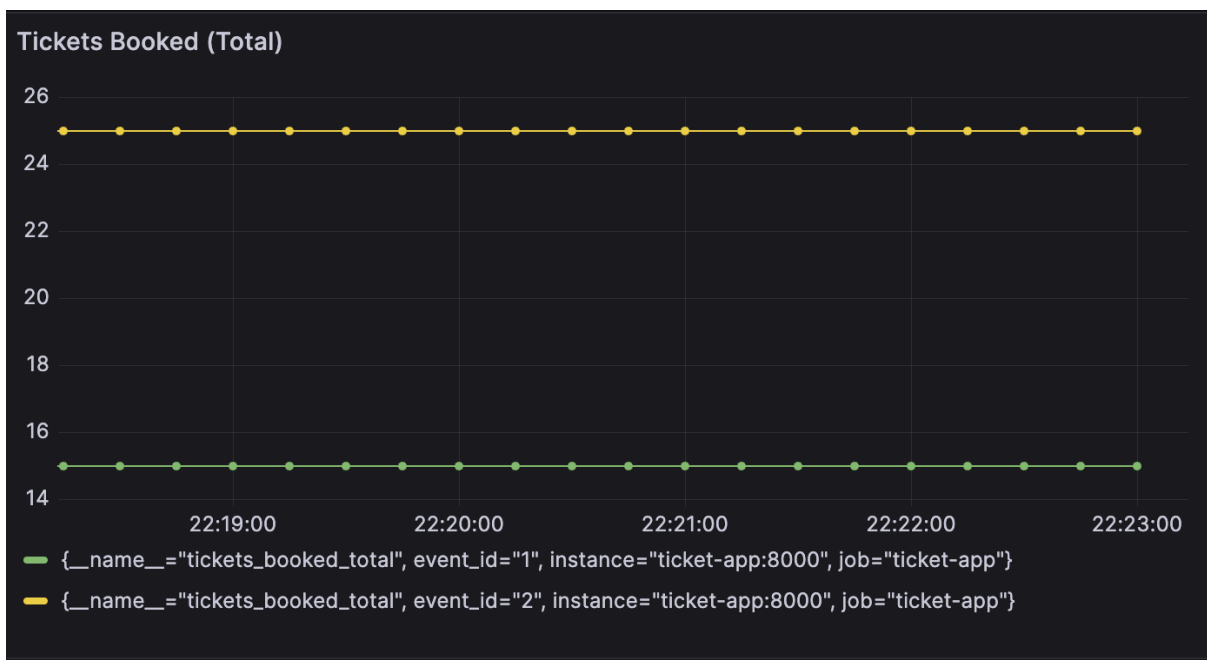
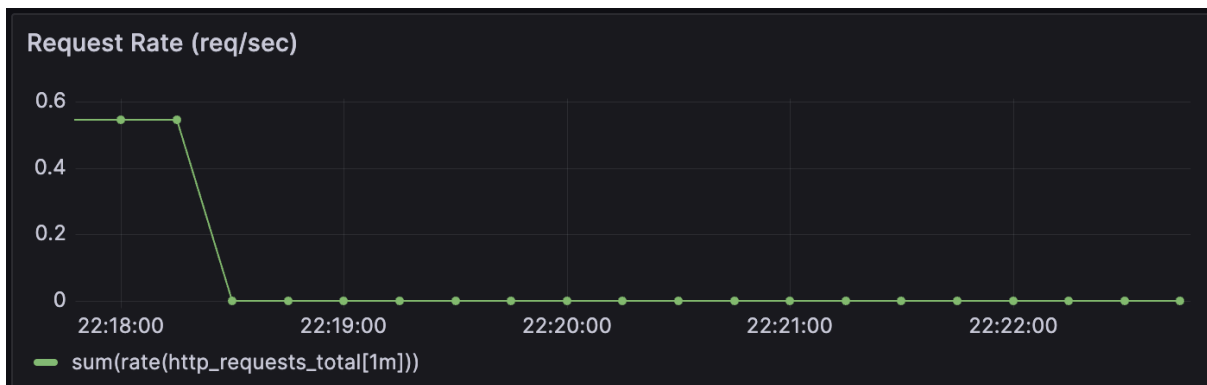
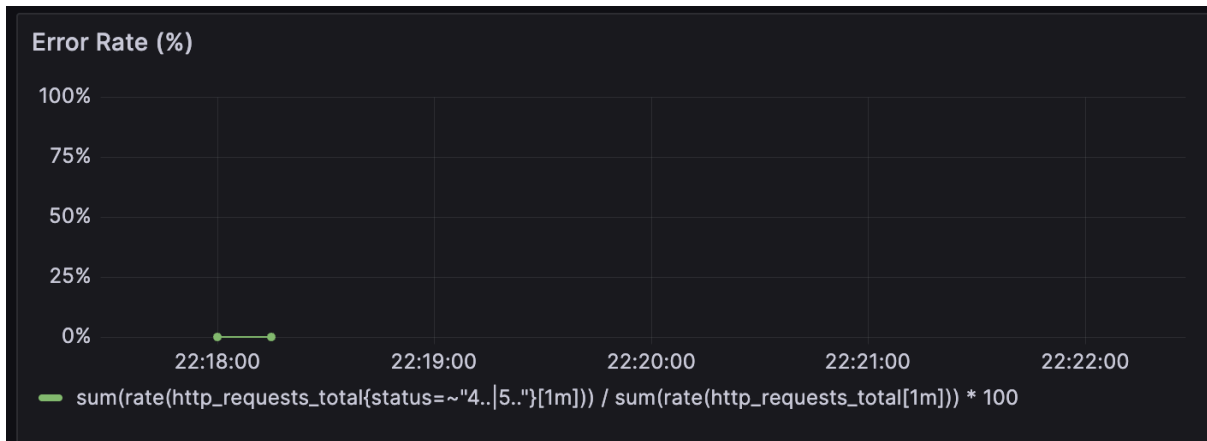


Avg Booking Time (ms)



Latency P95 (ms)





Machine measured: Node Exporter reads the Docker Desktop Linux VM running on the developer's MacBook Air (8 GB RAM), as exposed to the node-exporter container.

Self-explored metric/chart: the `demo_requests_total` counter and its cardinality behaviour (see Part E.2) were designed and explored independently of the class examples, to demonstrate why unbounded labels are a specific anti-pattern rather than just describing it abstractly.

Part C: Logs

What is logged, why, and where

The app writes structured JSON logs via a custom `JsonFormatter` (in `main.py`). Every log line includes time, service_name, severity, and message, plus contextual fields (event_id, seats, request_id) where relevant.

- **Successful bookings/cancellations** are logged at INFO level, inside `/book` and `/cancel/{id}`, right after the state change succeeds.
- **Failed requests** (event not found, insufficient seats, invalid input, booking not found, already cancelled) are logged at WARNING level, immediately before the corresponding `HTTPException` is raised. This was a deliberate addition after initial testing showed only successes were visible in Kibana, error visibility is necessary to diagnose real incidents.

A unique request_id (UUID) is generated once per request and attached to every log line for that request, whether it succeeds or fails, enabling exact traceability.

How Filebeat collects and parses logs

The app writes JSON to stdout. Docker captures this into a per-container log file. Filebeat (configured in `telemetry/filebeat/filebeat.yml`) watches Docker's container log directory, decodes each JSON line, and ships the parsed fields (not raw text) to Elasticsearch as structured fields, this is what makes severity, request_id, etc. individually searchable rather than buried in one text blob.

Storage and retention

Logs are stored in Elasticsearch, backed by a named Docker volume (`elasticsearch-data`). They survive normal container restarts (`docker compose restart / down without -v`), but are permanently deleted if the stack is torn down with `docker compose down -v`. No index lifecycle policy or automatic deletion is configured, logs accumulate until manually cleared.

Searching in Kibana

Example searches used during development and verification:

```
message : "Ticket booking successful"
message : "Booking failed"
severity : "WARNING"
request_id : "<uuid>"
```

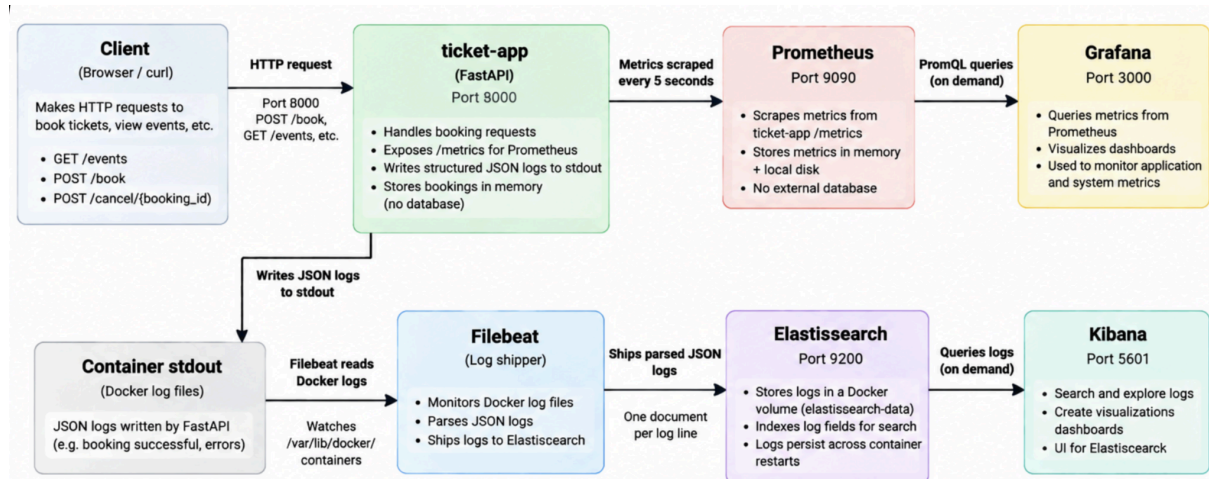
(Insert screenshot: one raw log document, expanded, showing all stored fields) (Insert screenshot: request_id search returning exactly one matching document)

Since all log fields are already valid JSON on write, no further parsing configuration was needed in Filebeat, this avoids the more error-prone plain-text regex parsing path.

Note on secrets/PII: no user credentials, payment details, or personal data are logged, only booking metadata (event ID, seat count, request ID).

Part D: System Design

Architecture Diagram



The system has two independent pipelines both originating at ticket-app:

Metrics path: ticket-app exposes /metrics. Prometheus scrapes it every 5 seconds and stores results in its own local storage (no external database). Grafana queries Prometheus on demand via PromQL whenever a dashboard is viewed.

Logging path: ticket-app writes JSON to stdout → Docker captures it to a log file → Filebeat reads and parses it → Elasticsearch stores it as a searchable document → Kibana queries Elasticsearch on demand.

Storage:

- ticket-app has no database; bookings live in memory and are lost on restart.
- Prometheus stores metrics in its own container, reset to zero on restart.
- Elasticsearch stores logs in a Docker volume, persisting across restarts but deleted with down -v.

Failure behaviour:

- **ticket-app down** → connection errors for users; no new metrics or logs generated; existing data unaffected.
- **Prometheus down** → Grafana shows "no data"; app keeps working; metrics generated during the outage are permanently lost (no buffering/backfill).
- **Elasticsearch down** → Filebeat queues logs locally and retries; no logs lost, only delayed; Kibana search unavailable until recovery.
- **Grafana down** → metrics still collected by Prometheus normally; only visualisation is affected.

Tracing one metric and one log

A single request, POST /book?event_id=1&seats=1, was traced through both pipelines.

Metric path: the counter `tickets_booked_total{event_id="1"}` incremented from 0 to 1 (having reset to 0 due to an earlier container rebuild — expected Prometheus counter behaviour). Within one scrape interval ($\leq 5s$), the new value appeared on the "Tickets Booked (Total)" Grafana panel as a step change.

Log path: the same request produced this log entry:

```
{
  "time": "2026-09-26T08:34:20.375733",
  "service_name": "ticket-app",
  "severity": "INFO",
  "message": "Ticket booking successful",
  "event_id": 1,
  "seats": 1,
  "request_id": "25bfa28c-3d47-49e0-9014-331dead1c85a"
}
```

Located via message : "Ticket booking successful" and confirmed unique via request_id : "25bfa28c-3d47-49e0-9014-331dead1c85a" in Kibana Discover.

GRAFANA:



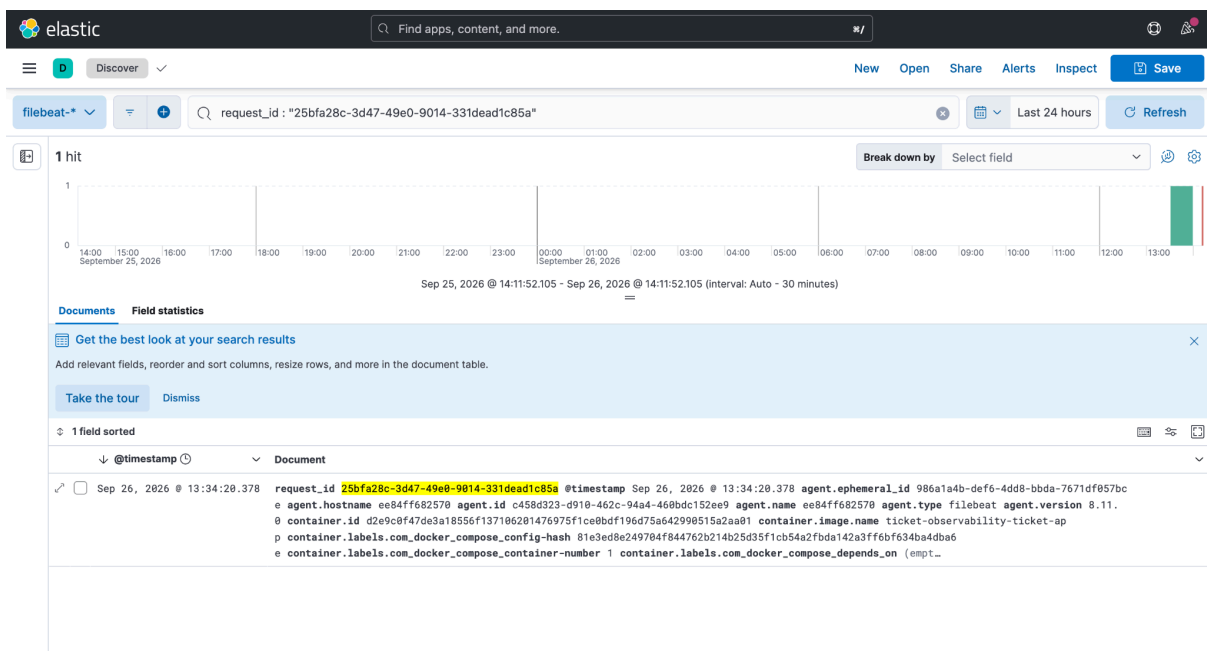
Grafana dashboard displaying the tickets_booked_total metric. The increase for event_id="1" shows that a successful booking request updated the application metric, which was then collected by Prometheus and visualized in Grafana.

PROMETHEUS:

```
{
  "@timestamp": "2026-09-26T08:34:20.378Z",
  "service_name": "ticket-app",
  "severity": "INFO",
  "message": "Ticket booking successful",
  "request_id": "25bfa28c-3d47-49e0-9014-331dead1c85a",
  "event_id": 1,
  "seats": 1,
  "agent.type": "filebeat",
  "container.name": "ticket-app"
}
```

A booking request generated a structured JSON log entry in the ticket application. The application recorded the booking event with fields including timestamp, service name, severity level, message, event ID, number of seats, and a unique request ID. Filebeat collected the container stdout log and forwarded it to Elasticsearch, where Kibana was used to search and inspect the stored fields.

KIBANA SEARCH:



Kibana Discover search showing retrieval of a specific booking request using request ID.

The shared `request_id` ties the metric update and the log entry back to the same booking, demonstrating that one user action is traceable across both pipelines independently.

Part E: Experiments

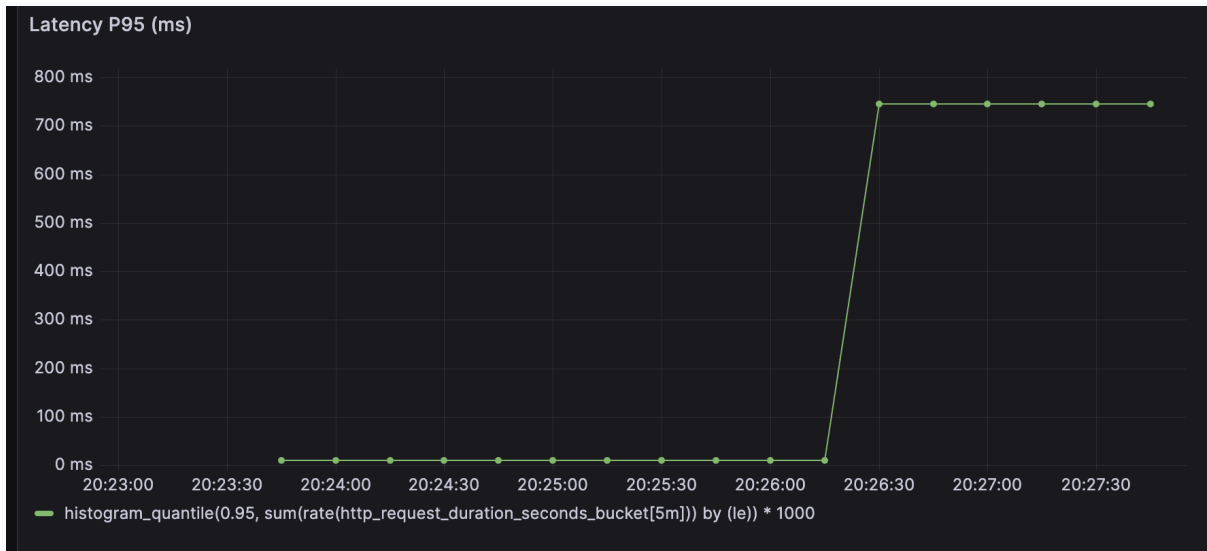
E1. Reproducing slow requests

Setup: POST /chaos/delay?enabled=true&delay=0.5 makes 20% of /book requests sleep 500ms before processing.

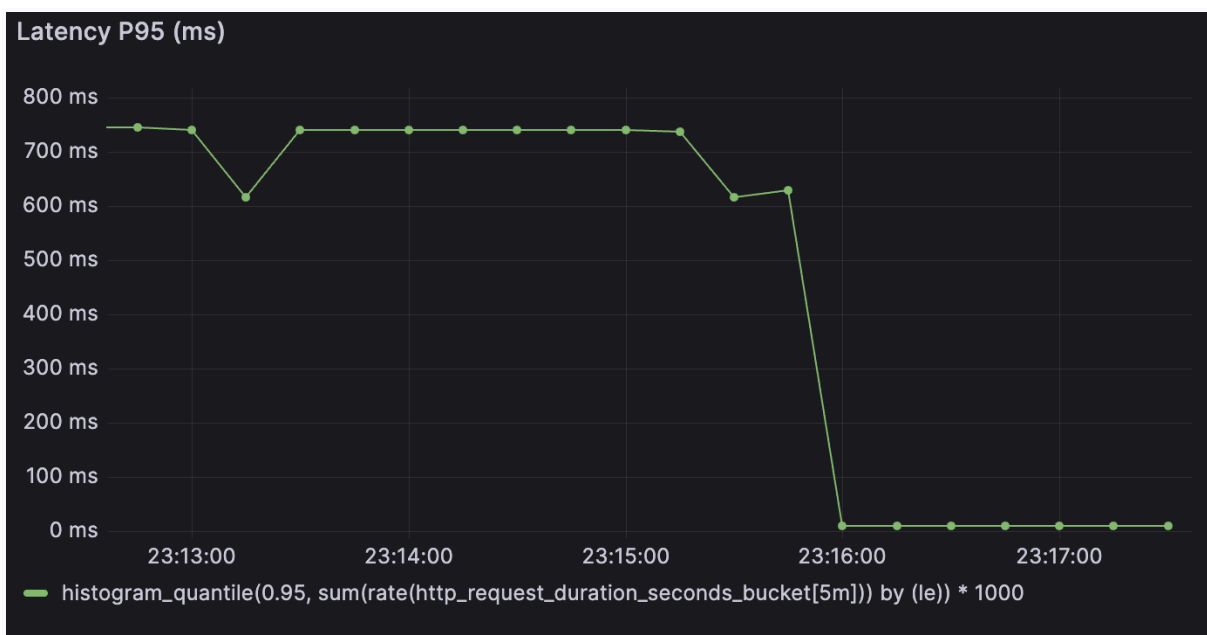
1. **Baseline:** 30 requests with chaos disabled. P95 latency held flat at ~9-10ms.
2. **Prediction:** with 20% of requests delayed 500ms (above the 5% P95 threshold), P95 was predicted to rise sharply into the hundreds of ms.
3. **Injection:** 30 requests with chaos enabled. P95 jumped from ~9ms to **~730ms**.
4. **Explanation:** the result matched the prediction. Since more than 5% of requests were delayed, the delayed requests dominated the P95 calculation. Roughly 1 in 5 booking attempts would feel noticeably slow to a user, despite zero errors, illustrating why latency percentiles matter independently of error rate.
5. **Recovery:** chaos disabled, 30 more requests sent. P95 returned to baseline (~9.5-9.8ms), confirming full recovery.



Baseline: Chaos delay disabled. 30 booking requests were executed. P95 latency remained stable at approximately 9-10ms.



Injection: After enabling the 500 ms artificial delay on 20% of requests, P95 latency increased sharply to approximately 730–740 ms, showing the impact of slow dependencies on user response time.



Recovery: After removing the injected delay, P95 latency returned to baseline values (~10 ms), confirming that the service recovered successfully after the fault was removed.

E2. Cardinality explosion

Setup: `demo_requests_total{request_id=...}` uses a fresh UUID label per request, deliberately unbounded, unlike bounded labels such as `event_id` (only 3 possible values).

1. With the bomb enabled, `count(demo_requests_total)` showed **1** after one request.
2. After 99 more requests (100 total), the count showed exactly **100**, one series per request, confirmed via `count(demo_requests_total)`.

3. The bomb was disabled and ticket-app restarted, clearing Prometheus's in-memory series. After 10 more (bomb-off) requests, `count(demo_requests_total)` returned **0 / empty**, confirming no new series were created once the unbounded label source was removed.
4. **Cost at scale:** 100 requests already created 100 permanent time series; a production system with thousands of daily requests would create a proportional number continuously, eventually exhausting Prometheus's memory. This is why request-level identifiers belong in logs (individually stored documents, as in Part C), not metric labels (which multiply the number of stored series). Bounded labels like `event_id` don't have this problem, since they map to a small, fixed set of values.

